Programmieren in C

Martin Zeller
Hochschule Ravensburg-Weingarten

Raum: T 110, Tel. 9760
E-Mail: zeller@hs-weingarten.de
martin-zeller@gmx.net

Tagesordnung

- Organisatorisches
- Ziel der Vorlesung
- Überblick „Programmieren“

Erwartung an Hörer (1)

- Mitdenken, Kenntnisse prüfen
- Nachfragen bei Unklarheiten
- selbständig Arbeiten (Literatur)
- Tutoren und Assistenten ansprechen
- Lerngruppen bilden und „leben“
- Übungen vorbereiten

- üben, üben, üben

Erwartung an Hörer (2)

- Keine Störungen
  - keine Nebendiskussionen
  - nicht essen
  - Laptops nur zum Mitschreiben
    (Papier ist m.E. besser)

Erwartung an Vortragenden

- Grundkenntisse Programmieren vermitteln
- Kontext herstellen
- Sinnhaftigkeit vermitteln
- Ansprechbar sein
-
-
-

Aktuelles

- Seminar Einführung in das Studium /Hüttenwochenende www.hs-weingarten.de/~huette

- Änderungen im Stundenplan, Ersatztermin
  - . . . .

Ziel der Vorlesung

Grundkenntisse in Programmieren vermitteln

- Syntax und Semantik von C
- Programmier-Stil
- Vorgehensweise der Software-Erstellung (Ansätze)
- Beispiele für Algorithmen
- Darstellung von Daten im Rechner (Ansätze)
- Programmierumgebung (z.Zt. Linux)

Inhalt der Vorlesung

- Ein- und Ausgabe: Tastatur und Bildschirm
- Zahlen, Berechnungen
- Verzweigungen
- Schleifen
- Arrays und Strings
- Entwurf und Dokumentation
- Funktionen
- Sichtbarkeit von Variablen

Inhalt der Vorlesung

- Programmierstil
- Pointer
- Strukturierte Datentypen
- Dynamische Speicherverwaltung
- Zugriff auf Dateien
- Rekursive Funktionen
- Aufzählungstypen/enumerations
- Präprozessoranweisungen

Quellenangaben

Die in der Vorlesung ausgegebenen Unterlagen ersetzen kein Buch.

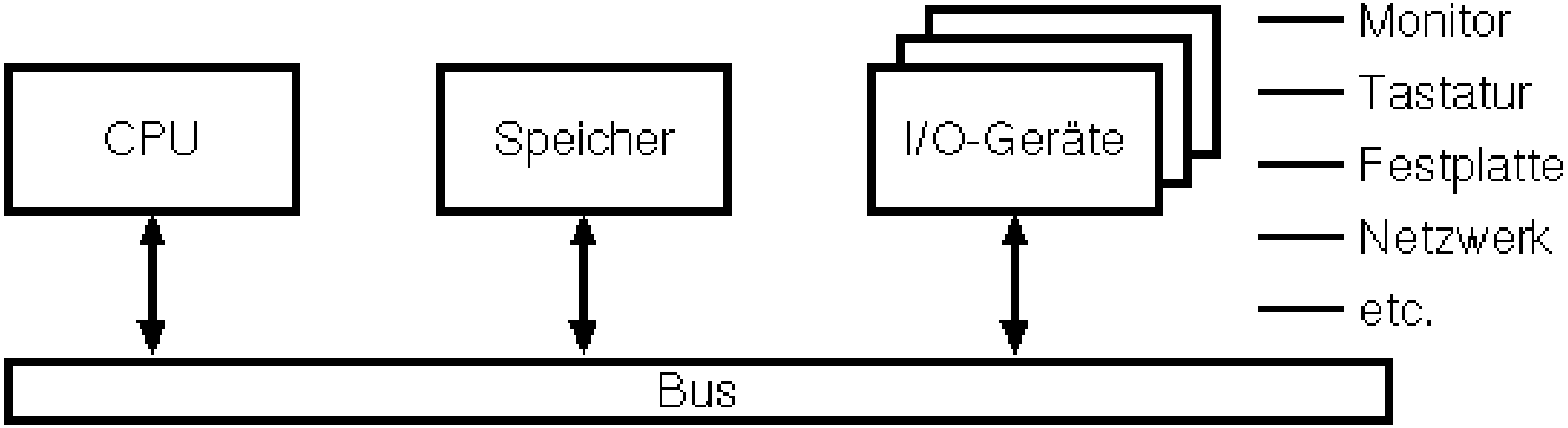
- Wolf, J.: C von A bis Z. Galileo Press, 2009 auch online verfügbar
- Darnell, Margolis: C A Software Engineering Approach. Springer Verlag, 1996
- Schildt, H.: C: The Complete Reference. Osborne McGraw-Hill, 2000.

Einführung

- Aufbau eines Entwicklungs-Computers:
  - Hardware
  - Betriebssystem
  - Programmierumgebung

Rechnerarchitektur

- Die „von Neumann“ Rechnerarchitektur
- Gemeinsamer Speicher für Daten und Programme
- Kommunikation über einen gemeinsamen Bus



Betriebssystem/Systemsoftware

- Verwaltet bzw. koordiniert die verschieden Hardware-Komponenten
- Startet und beendet Programme
- Benutzerschnittstelle (Desktop, Shell, DOS-Eing.)
- Bibliotheken, Komponenten
- Entwicklungs-Tools (Editor, Compiler, Linker)
- Dienstprogramme (Nutzerverwaltung, Netzwerk-Programme, etc.)

Programmierumgebung

- Einzelkomponenten
  - Editor
  - Compiler
  - Linker
  - Bibliotheken
  - Debugger, Versionskontroll-Tool, Test-Tool, etc.
- IDE (Integrated Development Environment)
  Alle Komponenten unter einer GUI
  (Graphical User Interface)

Arbeitsschritte

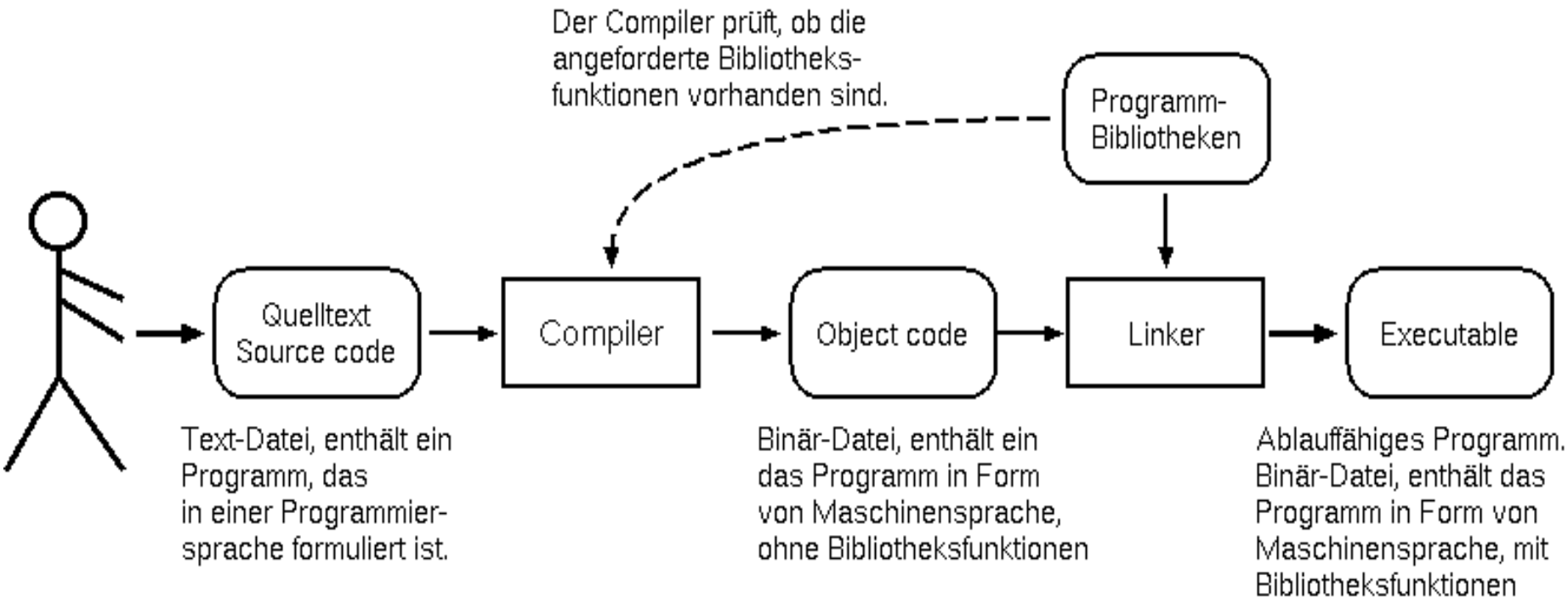
(1) Aufgabenstellung analysieren

(2) Problemlösung entwerfen (ggf. zurück zu (1))

(3) Programm erstellen (ggf. zurück zu (2) oder (1))

(4) Progamm testen
(ggf. zurück zu (3), (2) oder (1))

Programmieren i.e. Sinne



Ein Mini-Programm

```
/*****************************************************
 * Modul hello.c : Ein sehr schlichtes C-Programm
 * Aufgabe: Das Programm gibt eine Begruessung auf
 *          dem Bildschirm aus
 *****************************************************/
/*
 * Die Bibliothek stdio.h enthaelt Funktionen fuer
 * die Ein- und Ausgabe von Daten; hier: printf()
 * (stdio steht fuer standard input output)
 */

#include <stdio.h>                          /* 01 */
int main(void){                             /* 02 */
    printf("\n  Hello World \n");           /* 03 */
    printf("  Aller Anfang ... \n\n");      /* 04 */
    return 0;                               /* 05 */
}                                           /* 06 */
```

Ein- und Ausgabe

```
#include <stdio.h>
int main(void){
    int sum;
    int fix_number = 12345;                          /* 01 */
    int input_number;                                /* 02 */

    printf("\n fix_number: %d \n", fix_number);      /* 03 */
    printf(" input_number: %d \n", input_number);

    printf(" Bitte Zahl eingeben: ");
    scanf("%d", &input_number);                      /* 04 */
    printf(" input_number: %d \n", input_number);

    sum = input_number + fix_number;                 /* 05 */
    printf(" Summe: %d \n", sum);
    return 0;
}
```

Programm-Aufrufe

```
>a.out
 fix_number: 12345
 input_number: 1073795440
 Bitte Zahl eingeben: 4321
 input_number: 4321
 Summe: 16666

>a.out
 fix_number: 12345
 input_number: 1073795440
 Bitte Zahl eingeben: -123
 input_number: -123
 Summe: 12222

>a.out
 fix_number: 12345
 input_number: 1073795440
 Bitte Zahl eingeben: xyz
 input_number: 1073795440
 Summe: 1073807785
```

Variable für Zahlen

Der Typ einer Variablen bestimmt, wie groß der Speicherplatz der Variable ist und wie der Inhalt der Variable interpretiert wird.

Bsp.:

```
int number;
```

Diese Anweisung vereinbart eine **Variable** mit **Bezeichner** 'number' und dem **Datentyp** 'int'.

Der Inhalt der Variable 'number' wird als ganze Zahl interpretiert. (In arithmetischen und logischen Ausdrücken)

Variable für Zahlen

Jede Variable muss mit einem Wert belegt werden, bevor sie lesend verwendet wird.

Bsp.:

```
int number;
```

Diese Anweisung weist der Variable 'number' keinen Wert zu. Der Wert der Variablen ist daher quasi zufällig (s. Bsp. oben).

Deklaration mit Initialisierung:

```
int number = 7;

int num_count = number * 4;
```

Berechungen

Beispiel für eine Berechnung:

```
sum = input_number + fix_number;
```

Allgemeine Form der Zuweisung:

Variable = Ausdruck;

Ein Ausdruck kann u.a. eine Variable sein, eine Konstante oder ein arithmetischer bzw. ein logischer Ausdruck.

```
result = number;
result = number * (number + 1) – number/2;
```

Programm mit Schleife

```
#include <stdio.h>
int main(void){
    int limit;
    int i = 0;

    printf("\n Bitte obere Grenze eingeben: “);
    Scanf(“ %d“, &limit);

    while (i < limit){
        printf(" Zahl i = %d\n", i);
         i = i + 4;
    }
    printf(" Zahl i = %d\n", i);
    return 0;
}
```

Programm mit For-Schleife

```
#include <stdio.h>
int main(void){
    int sum   =  0;
    int loops = 10;
    int i;

    printf("\n Start Summieren mit %d\n", sum);

    for(i = 0; i < loops; i = i + 1){
        sum = sum + i;
        printf(" Summe = %d \n", sum);
    }
    return 0;
}
```

Elementare Datentypen

Typ-Name	üblicher(!) Wertebereich	
char	ganze Zahl	-128 bis 127
short	ganze Zahl	-32768 bis 32767
int	ganze Zahl	-2^{31} bis $2^{31}-1$
long	ganze Zahl	-2^{31} bis $2^{31}-1$
unsigned char	ganze Zahl	0 bis 255
unsigned short	ganze Zahl	0 bis 65535
unsigned int	ganze Zahl	0 bis $2^{32}-1$
unsigned long	ganze Zahl	0 bis $2^{32}-1$
float	Kommazahl	$-3.4*10^{38}$ bis $3.4*10^{38}$
double	Kommazahl	$-1.7*10^{308}$ bis $1.7*10^{308}$
long double	Kommazahl	$-1.1*10^{4932}$ bis $1.1*10^{4932}$

Darstellung von Binärzahlen

2-er Komplement für ganze Zahlen mit n Bits

- Eine Zahl k mit $0 \leq k < 2^{n-1}$ wird als positive Zahl k interpretiert.
- Eine Zahl k mit $2^{n-1} \leq k < 2^{n}$ wird als (negative) Zahl $k-2^{n}$ interpretiert.

Darstellung von Binärzahlen

Umrechnung k $\leftrightarrow$ -k; beide Richtungen gleich!

1) Alle Bits invertieren
2) Zum Ergebnis 1 addieren, eventuellen Überlauf ignorieren

- Eindeutige Darstellung der 0 (alle Bits 0)
- Addition ist unabhängig vom Vorzeichen

Operatoren

C enthält u. a. folgenden Operatoren:

```
=    Zuweisung
+    Addition
-    Subtraktion
*    Multiplikation
/    Division
%    Modulo (Rest der ganzzahligen Division)

>    Vergleich auf größer
<    Vergleich auf kleiner
==   Vergleich auf gleich
!=   Vergleich auf ungleich
>=   Vergleich auf größer oder gleich
<=   Vergleich auf kleiner oder gleich
&&   Logische und-Verknüpfung von log. Ausdrücken
||   Logische oder-Verknüpfung von log. Ausdrücken
!    Logische Negation
```

Programm mit Schleife

```
#include <stdio.h>
int main(void){
    int number;

    printf("\n Bitte Anfangswert eingeben: ");
    scanf("%d", &number);

    while (number < 1000000){
       number = number * number;
       printf(" Zahl = %d\n", number);
    }
    printf(" Zahl = %d\n", number);
    return 0;
}
```

Ausgabe der while-Schleife

Anfangswert: 3
Ausgabe: 9, 81, 6561, 43046721

Anfangswert: 5
Ausgabe: 25, 625, 390625, -2030932031
-2052264063 , -1083564287,
781532673

Anfangswert: 4
Ausgabe: 16, 256, 65536, 0, 0, 0, . . .
. . . Endlos-Schleife

Verzweigung

```
#include <stdio.h>
int main(void){
   int number;

   printf("\n Bitte Zahl eingeben: ");
   scanf("%d", &number);

   if (number > 0){
      printf("\n Eingabe war positiv; Zahl: %d", number);
      printf("\n zweite Anweisung im Block \n");
      return 0;
   }
   if(number == 0){
      printf("\n Eingabe war null; Zahl: %d\n", number);
   }else{
      printf("\n Eingabe war negativ; Zahl: %d\n", number);
   }
   return 0;
}
```

Switch

```
#include <stdio.h>
int main(void){
    int number;

    printf("\n Bitte Zahl eingeben: ");
    scanf("%d", &number);

    switch (number){
     case 1:  printf(" Ausgang 1 \n");
              break;
     case 2:  printf(" Ausgang 2 \n");
              break;
     case 5:  printf(" Ausgang 5 \n");
              break;
     default: printf(" default Ausgang \n");
    }
    return 0;
}
```

Grundbausteine

- Variable: Bezeichner, Datentyp, Wert
- Ausdruck: Zahlen, Bezeichner, Operatoren
- Entscheidungen/Verzweigungen:
  if-, switch-Anweisung
- Schleifen: while- und for-Anweisung
- Eingabe-und Ausgabe-Funktionen:
  scanf() und printf()
- Kommentare /* */ und // in C99

Datentyp Array (1)

```
#include <stdio.h>
int main(void){
  const int length = 10;
  int numbers[length];
  int i;

  for(i = 0; i < length; i = i + 1){
    numbers[i] = i * i;
    printf("numbers[%d] = %d\n", i, numbers[i]);
  }

  return 0;
}
```

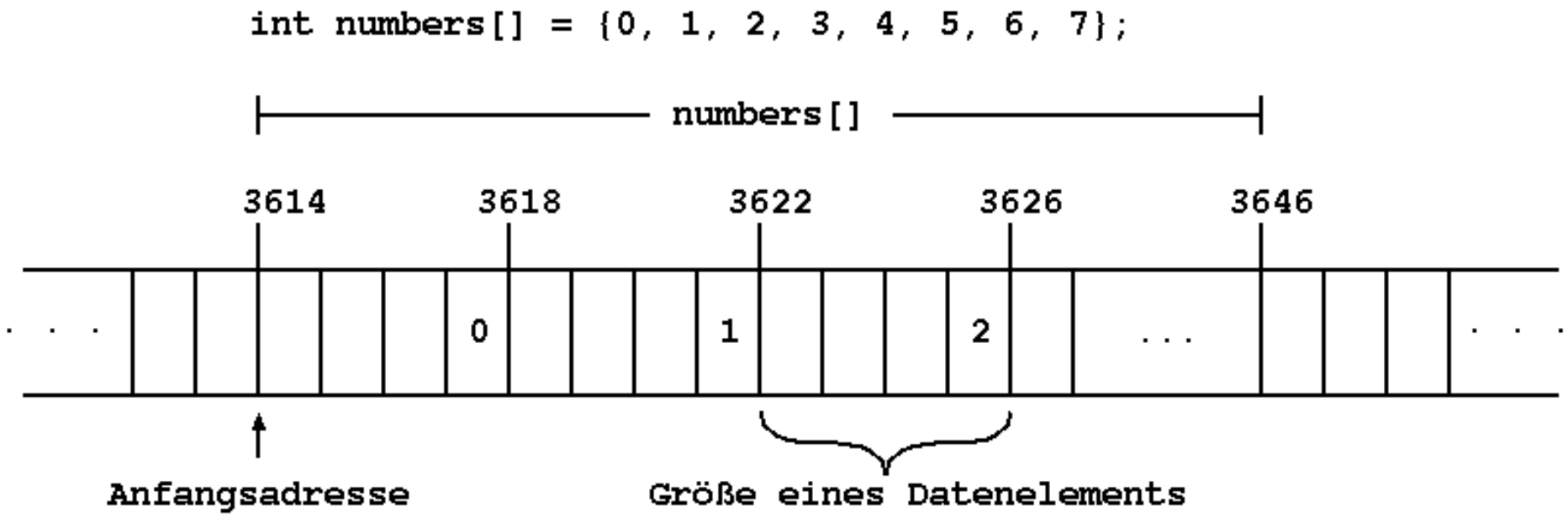
Datentyp Array (2)

- Eine Variable vom Typ Array speichert mehrere Werte eines Basis-Datentyps.
- Auf die einzelnen Werte des Basis-Datentyps wird durch einen Index zugegriffen.
- Erste Wert eines Arrays: Index = 0
  letzter Wert: Index = (Anzahl - 1)
- Die Sprache C überprüft nicht die Bereichsgrenzen. (i. Ggs. zu z.B. Java, Pascal)

Datentyp Array (3)

- Variable vom Typ Array können nicht als Ganzes zugewiesen oder verglichen werden
- Zuweisung/Vergleich der einzelnen Werte in einer Schleife
- Die Länge eines Array kann i.Allg. nicht aus der Array-Variable bestimmt werden.
- Die Länge eines Array kann über eine Konstante oder über eine Ende-Markierung bestimmt werden.

Datentyp Array (4)



Adresse von numbers[i]:

Anfangsadresse + i * Größe eines Elements

Array als String (1)

```
int main(void){
  const int length = 10;
  char  inString[length];
  int i;

  printf("\n Beliebige Eingabe mit <return> abschliessen:");
  scanf("%9s", inString);

  for(i = 0; i < length; i = i + 1){
    printf(" i = %d, inString[%d] = %c,", i, i, inString[i]);
    printf(" als int: inString[%d] = %d\n", i, inString[i]);
  }

  printf(" InputString: %s \n", inString);
  return 0;
}
```

Array als String (2)

```
Beliebige Eingabe mit <return> abschliessen:ABCDEFGHIJ
i = 0, inString[0] = A,  als int: inString[0] =  65
i = 1, inString[1] = B,  als int: inString[1] =  66
i = 2, inString[2] = C,  als int: inString[2] =  67
i = 3, inString[3] = D,  als int: inString[3] =  68
i = 4, inString[4] = E,  als int: inString[4] =  69
i = 5, inString[5] = F,  als int: inString[5] =  70
i = 6, inString[6] = G,  als int: inString[6] =  71
i = 7, inString[7] = H,  als int: inString[7] =  72
i = 8, inString[8] = I,  als int: inString[8] =  73
i = 9, inString[9] = ,  als int: inString[9] =   0
InputString: ABCDEFGHI
```

Array als String (3)

- Strings (Wörter bzw. Text) werden in C als Arrays mit Basis-Datentyp char realisiert.
- Das letzte Zeichen jedes Strings sollte/muss den Wert 0 haben.
  (Ende-Markierung von Strings in C)
- String-Funktionen ohne Längenbegrenzung öffnen Sicherheitslücken
  (z.B. gets(), strcpy(), . . .)
- Neuere Programmiersprachen unterstützen die Verarbeitung von Strings wesentlich besser.

Programmieren lernen

- Vorlesung: Programm => Aufgabe
- Übung: Aufgabe =>Programm
- Bausteine sammeln
- Zusammenbau üben

Algorithmen

Ein Algorithmus ist eine **Verfahrensvorschrift**, die eine Menge von **Objekten** durch **Operationen** aus einem definierten **Anfangszustand** in einen definierten **Endzustand** bringt.

- Kochrezept, Bastelanleitung
- Mathematische Definition (u.a. Turing-Maschine)

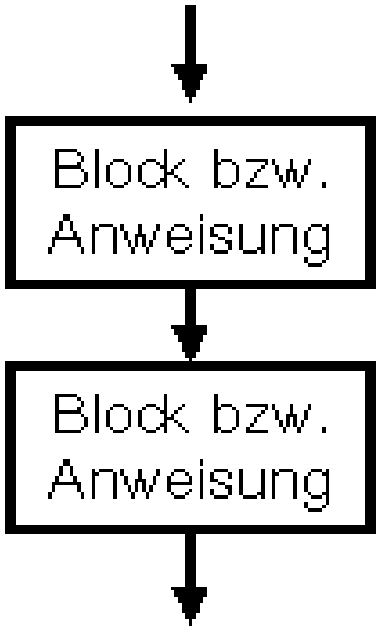
Beschreibung von Algorithmen

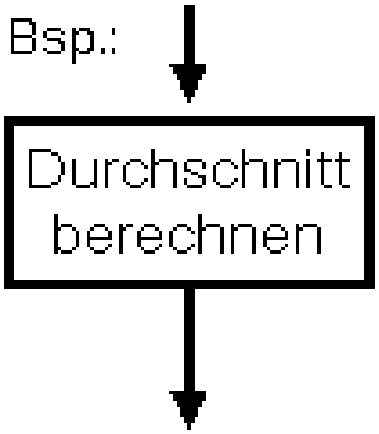
- Programmiersprache: C, Java, Basic, Pascal . . .
- Pseudo-Code (nicht standardisiert):

```
while budget > 0 do
   buy more things
endwhile
```

- Nassi-Shneiderman-Diagramm (Struktogramm)
- Ablaufdiagramm, Ablaufplan
- UML (Unified Modelling Language)
  s. Vorlesung Softwareengineering

Ablaufdiagramm
Block bzw. Sequenz





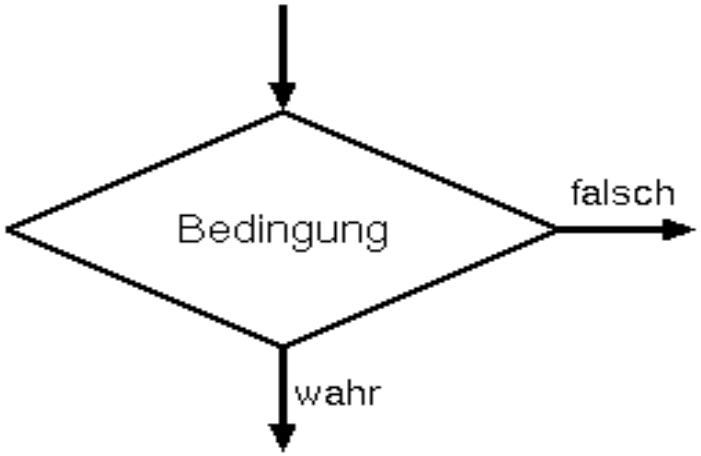
Pseudocode:

```
Anzahl der Zahlen berechnen
Zahlen aufsummieren
Summe durch Anzahl teilen
```

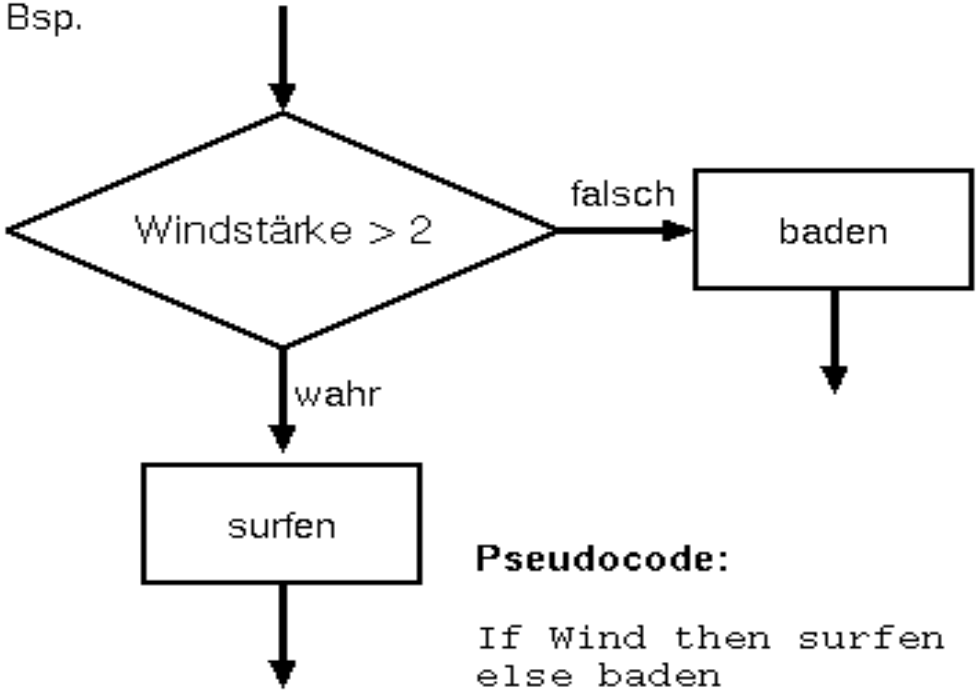
alternativ (abstraktere Sicht):

```
Durchschnitt der
Zahlen berechnen
```

Ablaufdiagramm, Verzweigung



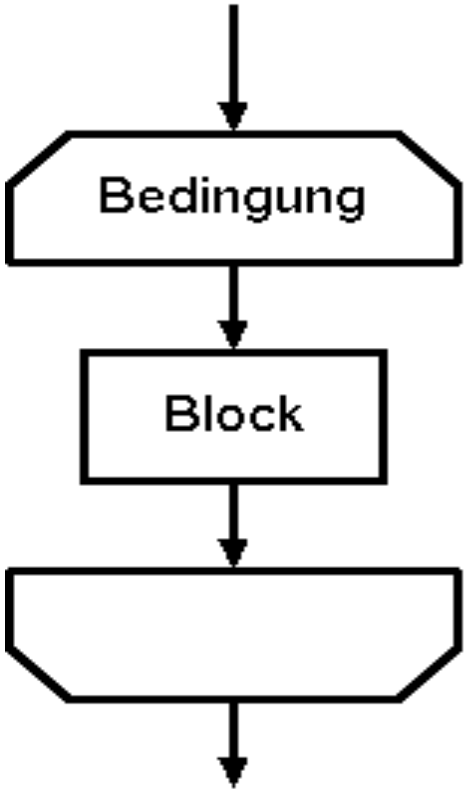
Bsp.

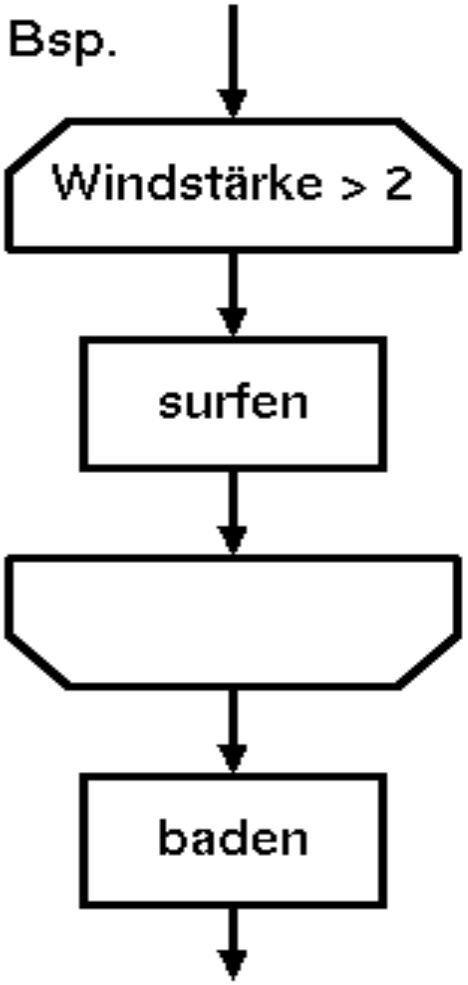


Pseudocode:

```
If Wind then surfen
else baden
```

Ablaufdiagramm, Schleife





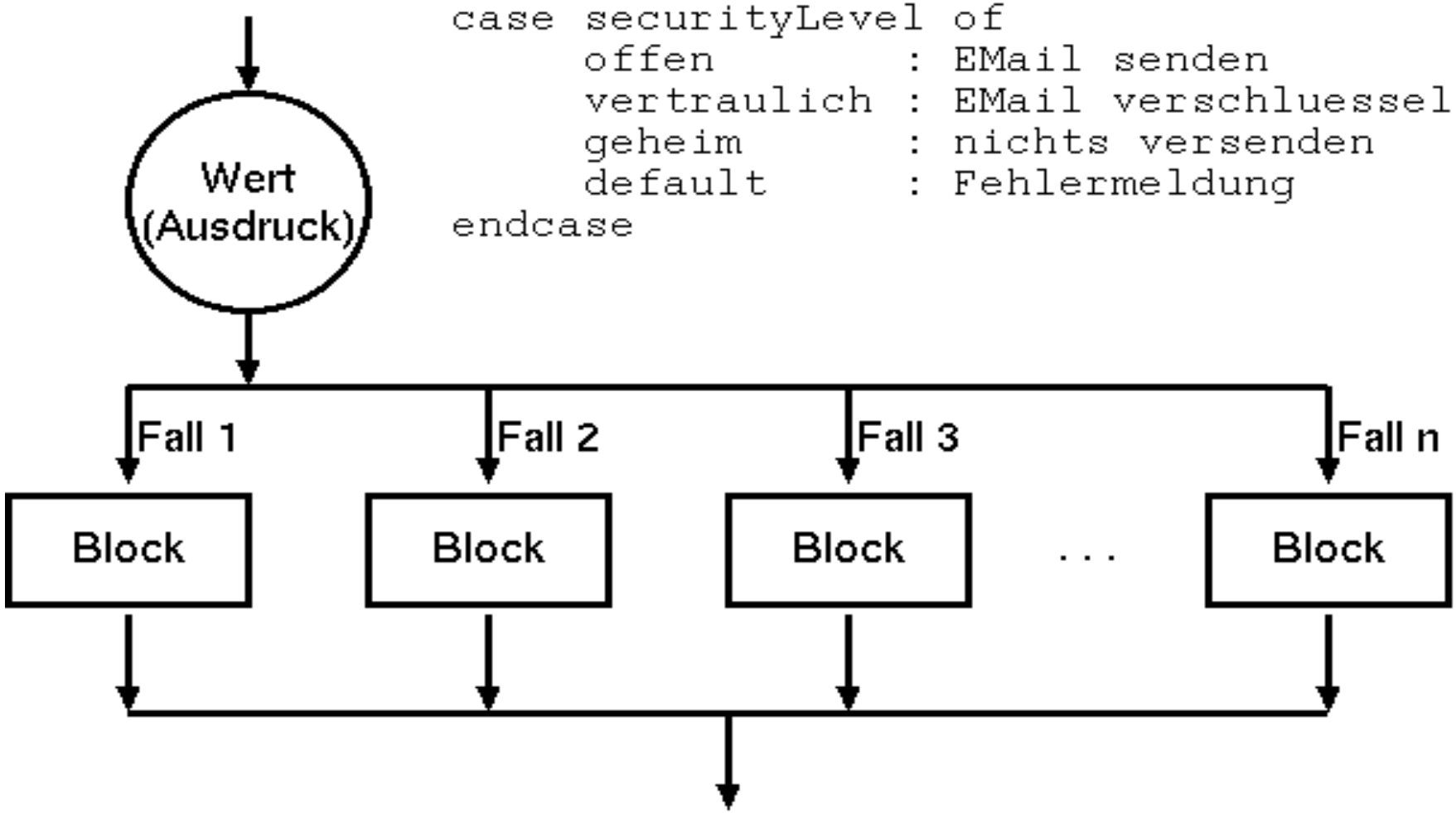
Pseudocode:

```
while wind do
    surfen
endwhile
baden
```

Mehrfachverzweigung (switch)

Pseudocode

```
case securityLevel of
     offen        : EMail senden
     vertraulich  : EMail verschluesseln
     geheim       : nichts versenden
     default      : Fehlermeldung
endcase
```



Elemente des Entwurfs

Prozedurale Systeme:

Ablaufdiagramme bzw. Pseudocode

- Sequenz von Verarbeitungsschritten
- Einfach- oder Mehrfachverzweigung
- Schleifen

Größere objektorientierte Systeme: UML

Regeln der strukturierten Programmierung

- Ein Strukturblock hat genau einen Eingang und genau einen Ausgang.
  (Mehrere Ausgänge können m.E. sinnvoll sein)
- Aneinanderreihen und Schachteln von Strukturblöcken ergibt wieder einen Strukturblock.
- Jeder Strukturblock muss mindestens einmal erreicht werden.
- Elementare Strukturblöcke sind Zuweisung und Funktionsaufruf.

Praktikum Programmieren

- Überarbeiten heißt i. Allg:
  Es sind nicht alle Fehler angestrichen. Bitte prüfen Sie selbst nochmals die gesamte Teilaufgabe.
- Ablaufdiagramm:
  Bitte zeichnen Sie sie Ablaufdiagramme mit den vorgegebenen Symbolen und sehr detailliert. Jede Variable muss dargestellt werden, ebenso Eingaben und Ausgaben.

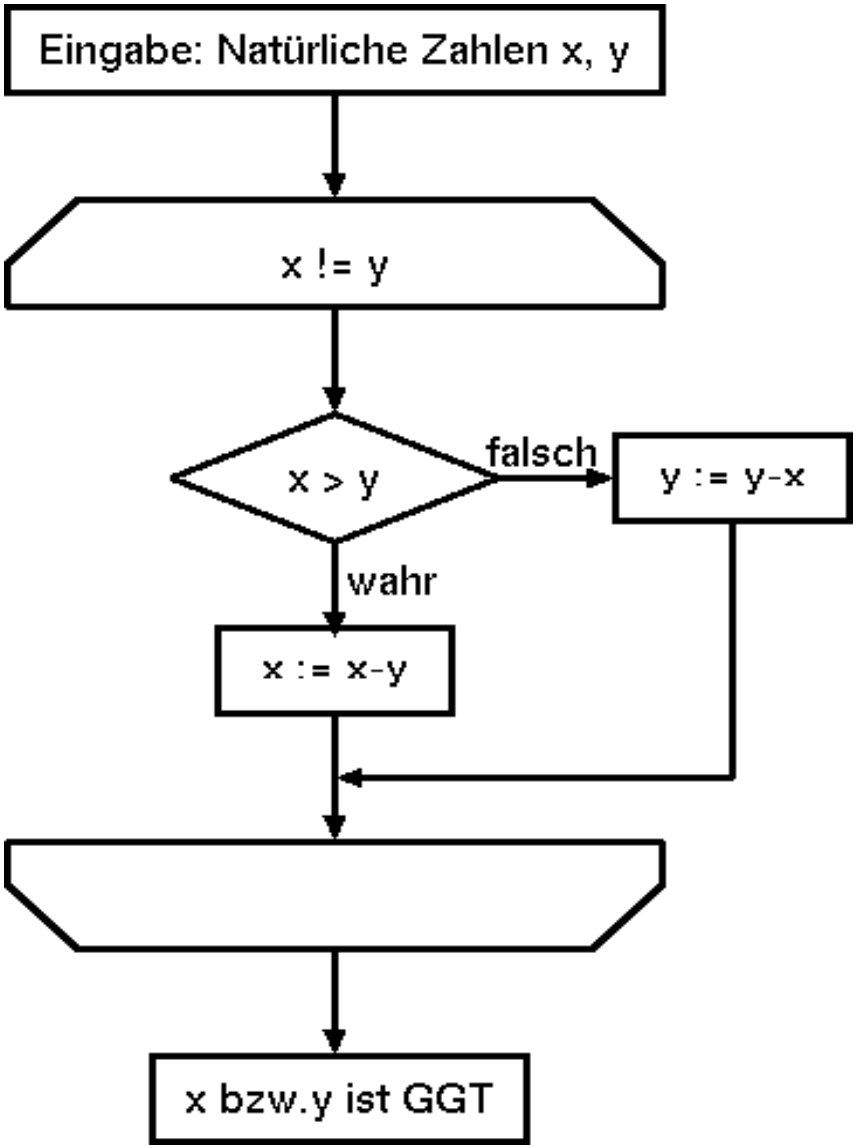
Größter Gemeinsamer Teiler Algorithmus: Euklid 300 v. Ch.

Gegeben: Zwei natürliche Zahlen x und y

Gesucht: Größter gemeinsamer Teiler von x, y

- Wiederhole bis x gleich y ist:
  - Wenn x größer als y ist: ziehe y von x ab und weise das Ergebnis x zu.
  - Wenn y größer als x ist: ziehe x von y ab und weise das Ergebnis y zu.
- Der Wert den x und y jetzt haben ist der größte gemeinsame Teiler.

GGT-Algorithmus Ablaufdiagramm



GGT: Ein C-Programm

```
int main(void){
    int zahl1    = 61*31*13*7;
    int zahl2    = 61*41*17*7;
    int current1 = zahl1;
    int current2 = zahl2;
    while (current1 != current2){
      if (current1 > current2){
         current1 = current1 - current2;
      }else{
         current2 = current2 - current1;
      }
    }
    printf("\n Der GGT der Zahlen %u und %u", zahl1, zahl2);
    printf(" lautet: %u \n\n", current1);
    return 0;
}
```

Funktionen

```
int ggt(int par1, int par2){
    while (par1 != par2){
       if (par1 > par2){
           par1 = par1 - par2;
       }else{
           par2 = par2 – par1;
       }
    }
    return par1;
}
int main(void){
    int num1    = 61*31*13*7;
    int num2    = 61*41*17*7;
    int ggt_result;

    ggt_result = ggt(num1, num2);

    printf("\n Der GGT der Zahlen %d und %d", num1, num2);
    printf(" lautet: %d \n\n", ggt_result);
    return 0;
}
```

Funktionen

```
long int pow(unsigned int base, unsigned int exponent){
   long int result = 1;
   int i;
   printf("\n Aufruf Funktion pow(%d, %u)", base, exponent);
   for(i = 0; i < exponent; i = i + 1){
      result = result * base;
   }
   return result;
}
int main(void){
   unsigned int b = 3;
   unsigned int result1;
   unsigned int result2;

   result1 = pow(b, 3);
   result2 = pow(result1, 2);

   printf("\n result1 = %u, " , result1);
   printf(" result2 = %u \n\n",  result2);
   return 0;
}
```

Funktionen

- Eine Funktion fasst mehrere Anweisungen zusammen, so dass diese Anweisungen mit einem Aufruf der Funktion ausgeführt werden.
- Eine Funktion benötigt i.a. Aufrufparameter und liefert i.a. einen Rückgabewert zurück.
- Bibliotheksfunktionen z.B. printf(), scanf()
- Eigene Funktionen (s. Übungen)

Prototyp einer Funktion

- Eine Funktion ist für den Nutzer durch folgende Attribute charakterisiert (Prototyp):
  - Name der Funktion (muss in C im gesamten Programm eindeutig sein)
  - Datentypen der Parameter, die Namen sind optional
  - Datentyp des Rückgabewerts
- Bsp.: `char []strncpy(char [], char [], int)` Prototyp der Funktion strncpy().

Signatur einer Funktion

- Die Signatur einer Funktion durch folgende Attribute gegeben:
  - Name der Funktion
  - Datentypen der Parameter,
    die Namen sind optional
- Bsp.: `strncpy(char [], char [], int)`
  Signatur der Funktion strncpy().

  Die Begriffe Prototyp und Signatur werden teilweise auch Synonym verwendet.

Implementierung einer Funktion

- Die Implementierung einer Funktion:
  - Name der Funktion
  - Namen und Datentypen der Parameter
  - Datentyp des Rückgabewerts
  - Ausführbarer Programmcode

Bsp.:

```
char[] strncpy(char s1[], char s2[], int n){
    int i = 0;
    while ((s2[i] != 0) && (i < n)){
        :
    }
}
```

Aufruf einer Funktion

1) Die Werte der aktuellen Parameter werden in die formalen Parameter kopiert.
   call-by-value
2) Der Programmcode der Funktion wird ausgeführt.
3) Der Rückgabewert wird zurückgegeben.
   D.h. der Funktionsaufruf wird durch den Rückgabewert ersetzt (innermost rewriting).

Arrays als Parameter

- Die Anfangsadresse des Arrays wird als Parameter übergeben.
- Die Anfangsadresse kann nicht verändert werden.
- Die einzelnen Elemente des Arrays können verändert werden (Call By Reference).

Sinn und Zweck von Funktionen

- Jeder Teil eines Programms soll einen treffenden Namen erhalten.
- Die Komplexität des Programms soll minimiert werden.
  (DRY – don't repeat yourself)
- Die Wartbarkeit des Programms wird verbessert:
  - Test
  - Modifikation
  - Kommentar bzw. Dokumentation

Sinn und Zweck von Funktionen

- Funktionen sollten kürzer als ca. 50 Zeilen sein (ohne Kommentar)
- Funktion sollten maximal 5 - 10 Variable enthalten
- Kontrollstrukturen sollten höchstens drei Stufen tief verschachtelt sein

Deklarationen

- Deklaration: Objekt (Funktion oder Variable) bekannt machen.
- Definition: Deklaration und Objekt anlegen. (Anweisungen ablegen, Speicher reservieren)

- Funktionen: Deklaration durch Prototyp
- Variablen: Deklaration durch **`extern`**
  z.B. **`extern int globalCount;`**
- Jedes deklarierte Objekt muss irgendwo im Programm definiert sein.

Funktionen verteilt auf Dateien (1)

```
/* Datei: ggtmain.c */
#include <stdio.h>
extern int ggt(unsigned int, unsigned int);

int main(void){
    int length = 5;
    int numbers[] = {2310, 14, 35, 70, 210};
    int i;
    int ggt_result = numbers[0];

    for(i = 1; i < length; i = i + 1){
       ggt_result = ggt(ggt_result, numbers[i]);
    }
    printf("\n Der GGT der Zahlen ");
    for(i = 0; i < length;i = i + 1){
       printf(" %u,", numbers[i]);
    }
    printf(" lautet: %u \n\n", ggt_result);
    return 0;
}
```

Funktionen verteilt auf Dateien (2)

```
/* Datei: ggtfun.c */

int ggt(int number1, int number2){

    while (number1 != number2){
       if (number1 > number2){
          number1 = number1 - number2;
       }else{
          number2 = number2 - number1;
       }
    }
    return number1;
}
```

Funktionen verteilt auf Dateien (3)

- Eine Funktion kann in einer eigenen Datei stehen.
- Eine Funktion kann nicht über mehrere Dateien verteilt sein.
- In einer Datei kann mehr als eine Funktion stehen.
- Eine Funktion muss dem Compiler erst bekannt sein, bevor sie verwendet (d.h. aufgerufen) werden kann. (Definition oder Deklaration durch Prototyp)

Funktionen verteilt auf Dateien (4)

- Header-Datei: name.h
  - Definition von Datentypen (später)
  - Deklaration von Funktionen (Prototypen)
- Programm-Datei: name.c
  - include der Header-Datei name.h
  - Definitionen/Implementierung der Funktionen
- Andere Programm-Dateien (xyz.c)
  - include der Header-Datei und Aufruf der Funktionen

Funktionen verteilt auf Dateien (5)

- Modul:
  Header-Datei und Implementierungs-Datei

Ziel der Verteilung auf Dateien

- Parallele Bearbeitung von Programmteilen
- Inkrementelles Compilieren
- Abspaltung von Bibliotheken;
  ggf. „shared“ (zur Laufzeit)

Sichtbarkeit von Variablen

```
int foo(int x, int b){
    b = 2 * x + b;
    return b;
}

int main(void){
    int b = 1;
    int c = 2;
    int result;

    if (b < c){
       int b = 3;
       result = foo(b, c);
    }
    printf("\n b = %d, result = %d, \n ", b, result);
    return 0;
}
```

Sichtbarkeit von Variablen

- Ist in einem Block ein innerer Block eingeschachtelt, so sind die Variablen des äußeren Blocks im inneren sichtbar (aber nicht umgekehrt).

- Ist im inneren Block eine Variable definiert, die den gleichen Namen besitzt, wie eine Variable des äußeren Blocks, so überdeckt die innere Variable die äußere.

Sichtbarkeit von Variablen

- Ein formaler Parameter einer Funktion erhält den **Wert** der beim Aufruf eingesetzten Variablen bzw. des eingesetzten Ausdrucks (call by value).
- Ein formaler Parameter einer Funktion ist nur innerhalb der Funktion sichtbar.
- Die beim Aufruf eingesetzte Variable wird durch die Funktion nicht verändert.
  (Arrays: Parameter ist die Adresse des Arrays. Die Adresse kann nicht verändert werden, der Inhalt der Array-Elemente sehr wohl: call by reference)

Globale Variablen

```
/* Datei useFoo.c */
#include <stdio.h>

int global = 0; /* define global variable */

int main(void){
    int result = 0;
    int i;

    for(i = 0; i < 4; i = i + 1){
       result = foo(i);
       global = global + 1;
       printf("  result = %d, \n", result);
    }
    return 0;
}
```

Globale Variablen

```
/* Datei defineFoo.c */

#include <stdio.h>

extern int global; /* declare global variable */

int foo(int x){
    x = x * global;    /* use global variable */
    return x;
}
```

Überdeckung von Variablen

```
const int x = 1;

void printX(){
  printf("\n in printX(): x = %d \n\n", x);
  return;
}

int main(void){
  int x = 2;
  int i;
  for(i = 0; i < 2; i = i + 1){
    int x = 3;
    printf("\n in for(){...}: x = %d", x);
  }
  printf("\n in main(){...}: x = %d", x);
  printX();
  return 0;
}
```

Globale Variablen und Konstanten

- Globale Variable sollten möglichst nicht verwendet werden, da sie die Wartbarkeit des Programms verschlechtern.
- Konstanten, die in mehr als einer Funktion verwendet werden, sollten dagegen global definiert werden (z.B. mathematische Konstanten oder auch programmspezifische Konstanten).

```
const double pi = 3.1415 . . . ;
const int lineLength = 128;
```

Globale Variable mit File Scope

```
static int global = 0;

int foo(int x){
    x = x * global;
    return x;
}

int main(void){
    int result = 0;
    int i;
    for(i = 0; i < 4; i = i + 1){
       result = foo(i);
       global = global + 1;
       printf("  result = %d, \n", result);
    }
    return 0;
}
```

Sichtbarkeit von Variablen

- Eine Variable, die nicht innerhalb eines Blocks deklariert ist, ist eine globale Variable.
- Eine globale Variable, die **static** deklariert ist, ist in der gesamten Datei sichtbar.
- Eine globale Variable, die **nicht static** deklariert ist, ist im gesamten Programm sichtbar.

Lebensdauer von Variablen

- Die Lebensdauer einer Variablen, die nicht „static“ deklariert wurde, ist auf den Block begrenzt, in dem sie deklariert wurde. (Wie Sichtbarkeit)

- Die Lebensdauer einer „static“ deklarierten Variablen ist auf die Programmlaufzeit begrenzt. D.h. der Wert der Variablen bleibt erhalten, auch wenn das Programm den Block verlässt, in dem die Variable deklariert wurde.

Lebensdauer von Variablen

- Eine Variable mit Initialisierung, die nicht „static“ deklariert wurde, wird jedesmal neu initialisiert, wenn „ihr“ Block erreicht wird.
- Eine „static“ deklarierte Variablen mit Initialisierung wird nur einmal - beim Programmstart - initialisiert.

Implizite Typ-Konvertierung (1)

- Der Datentyp eines Ausdrucks ergibt sich aus folgenden Regeln:
- Variable vom Typ char bzw. short werden in einem Ausdruck wie folgt konvertiert:

Ausgangs-Typ	Ziel-Typ	
`(signed) char`	`int`	
`unsigned char`	`int`	
`(signed) short`	`int`	
`unsigned short`	`unsigned int`	`(Falls short = int)`
`unsigned short`	`int`	`(sonst)`

Implizite Typ-Konvertierung (2)

In C sind die elementaren Datentypen wie folgt geordnet:

```
int < unsigned int < long int < unsigned long
unsigned long < float < double < long double
```

Werden zwei Ausdrücke durch einen arithmetischen Operator verknüpft, so ist der Typ des dadurch entstehenden Ausdruck der 'größere' der beiden Typen der beteiligten Ausdrücke.

Programmier-Stil

Stil: Durch spezifische Auswahl der lexikalischen und grammatikalischen Mittel gekennzeichnete Verwendungsweise der Sprache.

Ziel eines guten Programmier-Stil ist es, leicht verständliche Programme zu schreiben. (Stichwort Wartbarkeit)

Programmier-Stil (2)

Leitsatz: Prinzip der geringsten Überraschung

- Kommentare
- Bezeichner: Namen und Schreibweise von Variablen und Funktionen
- Text-Layout: Einrückungen und Klammern
- Schachtelungstiefe von Blöcken
- Implizite Typ-Konvertierung

Programmier-Stil (3)

Kommentar: Telegramm-Stil, Aktivsätze

- Module, Funktionen:
  - Sinn und Zweck, wie zu verwenden
  - Voraussetzungen, Grenzen, Ergebnis
  - Algorithmen und innerer Aufbau
- Blöcken, Zeilen:
  - Was soll erreicht werden
  - Ggf.: Wie wird das Ziel erreicht

Nur die nicht offensichtlichen Dinge kommentieren.

Programmier-Stil (4)

Einheitliche Bezeichner für Variable:

- Namen muss Inhalt bezeichnen, bevorzugt englisch
- Zusammengesetzte Namen:
  xx_yy (üblich in C) oder xxYy (üblich in C++, Java)

Einheitliche Bezeichner für Funktion:

- Standardname: anweisungObjekt(), z.B. drawLine().
- Wenn der Teil 'anweisung' offensichtlich ist, kann er entfallen (z.B. mean(...) statt computeMean(...))
- Wenn der Teil 'Objekt' offensichtlich ist, kann er entfallen (z.B. max(int x, int y) statt maxNumber(...))

Programmier-Stil (5)

Text-Layout: Einrückung, Klammern

```
Beginn eines Blocks{
   Anweisung1;
   Beginn eines geschachtelten Blocks{
      Beginn eines tiefer geschachtelten Blocks{
         Anweisung2;
         Anweisung3;
      }
      Anweisung4;
   }
   Anweisung5;
}
```

Programmier-Stil (6)

Schachtelungstiefe

- Normale Schachtelungstiefe: 2
- Nur in Sonderfällen 3 oder evtl. mehr

Komplexer Blöcke können in Funktionen aufgespalten werden.

Programmier-Stil (7)

Implizite Typkonvertierung

- Nur für Typerweiterungen innerhalb von Ganzahl-Typen bzw. innerhalb von Gleitkomma-Typen

Alternativen:

- Explizite Konvertierung (cast), z.B. x = (float) y / z
- Funktionen (und cast), z.B. n = (int) floor(y * z + 0.5)

Programmier-Stil (8)

Sichtbarkeit von Variablen

- Variablen sollen nur in dem Block sichtbar sein, in dem sie benötigt werden (d.h. möglichst lokal).
- Variablen sollen so benannt werden, dass sie sich nicht gegenseitig überdecken.
- Verwenden Sie keine globalen Variablen.

Programmier-Stil (9) Funktionen

- Eine Funktion soll eine klar abgegrenzte Aufgabe erfüllen.
  (**Eine** Sache **richtig** machen.)
- Eine Funktionen soll nur ein Ergebnis liefern und ggf. Aufrufparameter ändern (aber keine globalen Variablen).
- Eine Funktion sollte nicht länger als ca. 50 Zeilen sein (ohne Kommentar).
- Eine Funktion sollte maximal 5-10 Variable verwenden.

Zeiger/Pointer (1)

```
#include <stdio.h>
int main(void){
    float num = 1.234;
    float *numPtr;
    numPtr = &num;

    printf(" der Wert von num ist: %7.3f \n", num);
    printf(" die Adresse von num ist: %u \n", numPtr);
    printf(" Wert in Adresse von num: %7.3f \n", *numPtr);

    *numPtr = num + 4.321;

    printf("\n der Wert von num ist: %7.3f \n", num);
    printf(" die Adresse von num ist: %u \n", numPtr);
    printf(" Wert in Adresse von num: %7.3f \n", *numPtr);
    return 0;
}
```

Zeiger/Pointer (2)

`float size;`	“normale“ float-Variable
`float *numPtr;`	Zeiger (Pointer) auf eine float-Variable
`numPtr = &size;`	Der Pointer erhält als Wert die Adresse der Variablen size. Der Pointer zeigt nun auf size.

Zeiger/Pointer (3)

```
int height;
int width;
int *numPtr1;
int *numPtr2 = NULL;
numPtr1 = &height;
*numPtr1 = 42;
width = *numPtr1;
numPtr2 = numPtr1;
```

NULL ist eine Konstante

Die Variable, auf die numPtr1 zeigt, erhält den Wert 42 zugewiesen.

Die Variable erhält den Wert, auf den numPtr1 zeigt, zugewiesen.

Der Pointer numPtr2 zeigt jetzt ebenfalls auf die Variable height.

Call By Reference

```
void increment(int *num){
    *num = *num + 1;
    num = num + 1;
    return;
}

int main(void){
    int testNum  = 3;
    int *testNumPtr = &testNum;

    printf("\n Wert testNum: %d ",testNum);

    increment(testNumPtr);
    increment(&testNum);

    printf("\n Wert testNum: %d \n",testNum);
    return 0;
}
```

Call By Reference

- Pointer/Adresse als Übergabe-Parameter an Funktion
  - Die Funktion kann das Datenelement verändern
  - Die Änderungen sind außerhalb der Funktion sichtbar
- Den Pointer selbst kann die Funktion zwar ändern, diese Änderung wirkt sich aber nur innerhalb der Funktion aus. (Der Pointer wird bei der Übergabe kopiert.)

Pointer und Strings (1)

```
int main(void){
    char  lastName[] = "Mueller-Luedenscheid";
    char *anyText;
    anyText = lastName;

    printf(" lastName: %s \n", lastName);
    printf(" anyText:  %s \n", anyText);

    anyText[8] = 'M';
    printf(" lastName: %s \n", lastName);

    lastName[0] = 'L';
    printf(" anyText:  %s \n", anyText);

    anyText = anyText + 8;
    printf(" anyText:  %s \n", anyText);
    printf(" anyText als char:  %c \n", *anyText);
    return 0;
}
```

Pointer und Strings (2)

Das Programm erzeugt folgende Ausgabe:

```
lastName: Mueller-Luedenscheid
anyText:  Mueller-Luedenscheid
lastName: Mueller-Muedenscheid
anyText:  Lueller-Muedenscheid
anyText:  Muedenscheid
anyText als char:  M
```

Pointer und Strings (3)

```
char  lastName[]= "Mueller-Luedenscheid";
```

- Definiert einen Pointer auf char
- Reserviert Speicherplatz für 21 chars
- Vorbelegung mit „Mueller-L ... “

Der Pointer `lastName` zeigt auf den Anfang des reservierten Speicherplatzes. Er kann nicht verändert werden.

```
char *anyText;
```

Diese Anweisung definiert einen Pointer auf char. Es wird kein Speicherplatz reserviert, der Pointer kann verändert werden.

Pointer und Arrays (1)

```
<BasisTyp>  var[n];
```

Die Definition eines Arrays definiert einen Pointer auf den Anfang des Arrays und reserviert Platz für die einzelnen Elemente. Der Pointer kann nicht verändert werden.

```
<BasisTyp>  *var;
```

Die Definition eines Pointers reserviert keinen Speicherplatz. Der definierte Pointer kann verändert werden.

Pointer und Arrays(2)

```
int main(void){
    int  numbers[] = {0, 1, 2, 3, 4, 5, 6, 7};
    int *numPtr;
    numPtr = numbers;   /* gueltige Zuweisung */
    printf("  numPtr ist: %u \n", numPtr);
    printf(" *numPtr ist: %d \n", *numPtr);
    numPtr = numPtr + 1;      /* plus 4 (oder 8) */
    printf("  numPtr ist: %u \n", numPtr);
    printf(" *numPtr ist: %d \n", *numPtr);
    numPtr = numPtr + 3;      /* plus 12 (oder 24) */
    printf("  numPtr ist: %u \n", numPtr);
    printf(" *numPtr ist: %d \n", *numPtr);
    return 0;
}
```

Pointer und Arrays (3)

```
<BasisTyp>  *varPtr;
```

Ein Pointer kann inkrementiert und dekrementiert werden:

```
varPtr = varPtr + 1;
varPtr = varPtr - 1;
```

Belegt ein Element des BasisTyps im Speicher n Bytes, so werden tatsächlich folgende Operationen ausgeführt:

`varPtr = varPtr + 1;` ⇒ varPtr = varPtr + n

`varPtr = varPtr - 1;` ⇒ varPtr = varPtr – n

Der Pointer wird um n erhöht bzw. vermindert.

Pointer und Arrays (4)

```
<BasisTyp>  *varPtr;
```

Zu einem Pointer kann eine ganze Zahl addiert werden; ebenso kann von einem Pointer eine ganze Zahl abgezogen werden.

```
varPtr = varPtr + 3;
varPtr = varPtr - 5;
```

Belegt ein Element des BasisTyps im Speicher n Bytes, so werden tatsächlich folgende Operationen ausgeführt:

`varPtr = varPtr + k;` ⇒ varPtr = varPtr + n*k

`varPtr = varPtr - k;` ⇒ varPtr = varPtr – n*k

Der Pointer wird um n*k erhöht bzw. vermindert.

Pointer und Arrays (5)

- Ein Pointer kann durch die Operationen ++, --, +, - dazu gebracht werden, auf verschiedene Elemente eines Arrays zu zeigen.
  - Schneller als über Index (vielleicht)
  - Weniger leicht lesbar
  - Fehleranfälliger
- Empfehlung:
  Verzichten Sie auf Pointer-Arithmetik!

Strukturierte Datentypen (1)

```
int main(void){

    struct personId{
       char firstName[20];
       char lastName[30];
       short yearOfBirth;
    };
    struct personId  person1 = {"James", "Cook", 1728};
    struct personId  person2;

    person2 = person1;
    person2.yearOfBirth = 1699;

    printf(" pers1 lastName: %s \n", person1.lastName);
    printf(" pers2 firstName: %s\n", person2.firstName);
    printf(" pers2 borne in %4i\n", person2.yearOfBirth);
   return 0;
}
```

Strukturierte Datentypen (2)

Definition einer Struktur mit drei Komponenten.

```
struct personId{
      char firstName[20];
      char lastName[30];
      short yearOfBirth;
};
```

Definition einer Variablen vom Typ struct personId

```
struct personId person1;
```

Der Compiler reserviert Speicherplatz, aber die Komponenten der Struktur werden nicht initialisiert.

Strukturierte Datentypen (3)

Definition einer Variablen vom Typ struct personId

```
struct personId person1 = {"James", "Cook", 1728};
```

Der Compiler reserviert Speicherplatz, und initialisiert die Komponenten der Struktur.

Strukturierte Datentypen (4)

Definition einer Variablen vom Typ struct personId

```
struct personId person1 = {"James", "Cook", 1728};
```

Auf eine Komponenten der Struktur wird durch den Namen der Komponente zugegriffen:

```
person1.yearOfBirth = 1983;          /* schreibend */
if(person1.yearOfBirth < 2000){...   /* lesend     */
```

Strukturen vs. Arrays

- Strukturen fassen mehrere Datenelemente zusammen, die unterschiedliche Bedeutung tragen bzw. die unterschiedlich verwendet werden. Die Elemente können unterschiedliche Datentypen besitzen.

- Arrays fassen mehrere Datenelemente zusammen, die die gleiche Bedeutung tragen bzw. die einheitlich verwendet werden. Die Elemente besitzen alle denselben Datentyp.

Pointer auf Strukturen

Definition einer Struktur mit drei Komponenten.

```
struct personId{
    char firstName[20];
    char lastName[30];
    short yearOfBirth;
};
```

Variable vom Typ struct personId

```
struct personId person1;
```

Variable vom Typ Pointer auf struct personId

```
struct personId *personPtr;
```

Pointer auf Strukturen

Zuweisung

```
personPtr = &person1;
```

Zwei Möglichkeiten der Dereferenzierung

```
(*personPtr).yearOfBirth

personPtr->yearOfBirth
```

Beide Anweisungen greifen auf die Komponente `yearOfBirth` der Variablen zu, auf die `personPtr` zeigt.

Die zweite Schreibweise (->) ist zu bevorzugen.

Strukturierte Datentypen

Eine Struktur kann einen oder mehrere Pointer auf den eigenen Typ als Komponente enthalten.

```
struct personId{
          char firstName[20];
          char lastName[30];
          short yearOfBirth;
          struct personId *next;
        };
```

Die Komponente `next` der Struktur kann einen Pointer auf eine Variable vom Typ `struct personId` aufnehmen.

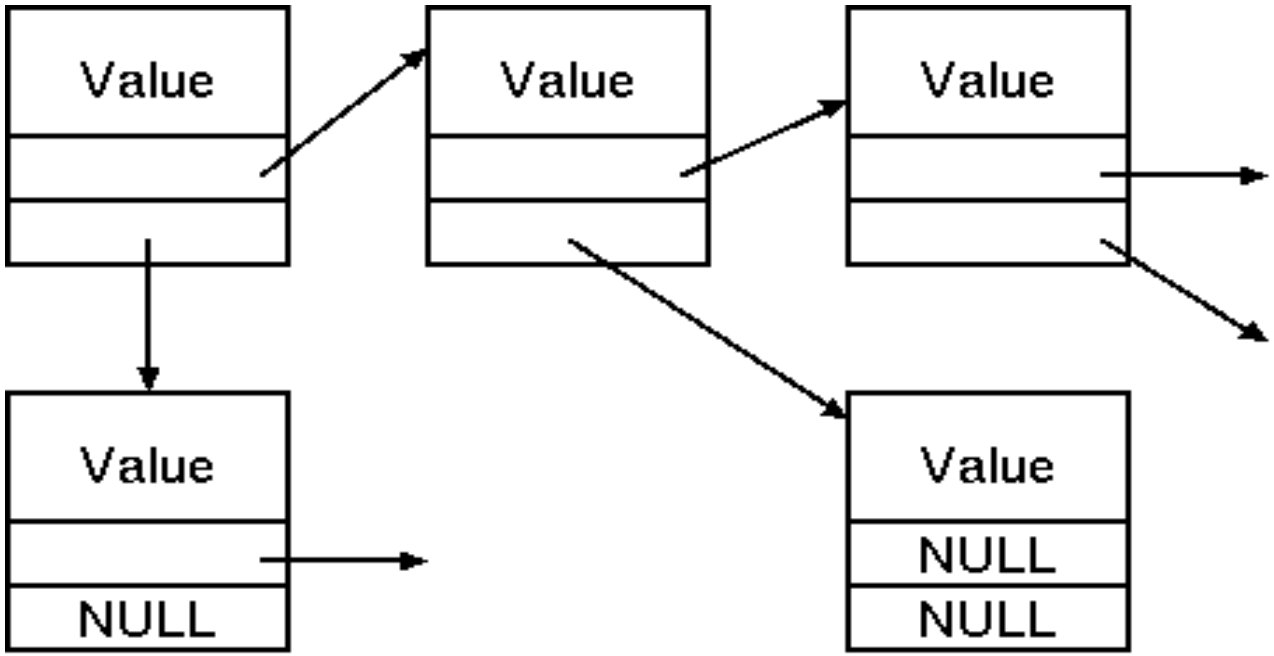
Pointer auf Strukturierte Datentypen

```
struct personId  person1 = {"James", "Cook", 1728, NULL};
struct personId  person2 = {"Captain", "Kirk", 8307, NULL};

person1.next = &person2;
```

```
 person1.next     ≙  &person2
*person1.next     ≙  person2
 person1.next->   ≙  person2.

 person1.next->lastname  ≙  person2.lastName
```

Strukturierte Datentypen (Ausblick)

Aus Strukturen, die ggf. mehrere Pointer auf den eigenen Typ als Komponente enthalten, können Listen, Bäume bzw, allg. Graphen aufgebaut werden.



Verschachtelte Strukturen

```
struct persIdentity{
    char  firstName[20];
    char  lastName[30];
    short yearOfBirth;
};

struct persAddress{
    char street[30];
    int  zipCode;
    char city[30];
};

struct person{
    struct persIdentity identity;
    struct persAddress address;
    struct person *next;
};
struct person  person1;
```

Verschachtelte Strukturen

```
struct person  person1;
struct persIdentity currentId;
currentId = person1.identity;
currentId.yearOfBirth = 1699;
strncpy(person1.address.street, "Rosenweg 3", 29);
```

Die Funktion `strncpy(char[], char[], int)` finden Sie in der Bibliothek `string.h`

Die Funktion kopiert den Inhalt des zweiten Arrays in das erste Array. Der letzte Parameter gibt an, wieviele Buchstaben maximal kopiert werden.

Verschachtelte Strukturen

Große Strukturen können/sollen durch Definition von Teilstrukturen unterteilt werden (wie Funktionen).

- Bessere Lesbarkeit
- Einfachere Handhabung

Prinzipien

- „teile und herrsche“
- Trennung der Belange (separation of concerns)

Typdefinitionen

```
struct persIdentity{
   char firstName[20];
   char lastName[30];
   short yearOfBirth;
};

struct persAddress{
   char street[30];
   int zipCode;
   char city[30];
};

typedef struct persIdentity PERS_ID;
typedef struct persAddress PERS_ADDRESS;

typedef struct person{      /* Definition der Struktur   */
   PERS_ID identity;        /* und Umbenennung in einem. */
   PERS_ADDRESS address;
   struct person *next;
} PERSON;
PERSON person1;
```

Typdefinitionen

```
struct persIdentity{
        ...
};
```

Definiert den Datentyp `struct persIdentity`

```
typedef struct persIdentity{
        ...
} PERS_ID;
```

Definiert den Datentyp `struct persIdentity` und den identischen Datentyp `PERS_ID`.

Durch `typedef` wird ein zweiter Name für einen bestehenden Datentyp definiert.

Typdefinitionen

Typdefinitionen können die Lesbarkeit eines Programms erheblich erhöhen.

Schwierig zu lesende Definition:

```
struct order *orders[10];
```

Klarere Definition:

```
typedef struct order ORDER;
typedef ORDER *ORDER_PTR;
ORDER_PTR orders[10];
```

Strukturen als Parameter

- Parameter vom Typ `struct xy` werden by-value übergeben
- Auch eingebettete Arrays werden kopiert
- Auch eingebettete Pointer werden vom aktuellen in den formalen Parameter kopiert. Ein Pointer zeigt dann auf dieselbe Variable wie im aktuellen Parameter.

Pointer in Strukturen

```
struct point{
    float x;
    float y;
};
typedef struct point POINT;
typedef POINT *POINT_PTR;

struct rectangle{
    POINT_PTR leftUpper;
    POINT_PTR rightLower;
};
typedef struct rectangle RECTANGLE;

float areaRectangle(RECTANGLE recta){
  float height = recta.leftUpper->y - . . .
  float width  = recta.rightLower->x - . . .
```

Dynamische Speicherverwaltung

- Bisher:
  - Alle verwendeten Variablen müssen zur Compile-Zeit deklariert sein.
- Neu:
  - Variable können auch zur Laufzeit neu angelegt und wieder gelöscht werden.
  - Diese Variablen können nicht über einen Namen, sondern nur über einen Pointer angesprochen werden.

Dynamische Speicherverwaltung

Ziel der dynamischen Speicherverwaltung

- Das Programm belegt nur soviel Speicher wie nötig
- Der Speicher wird nur so lange wie nötig belegt

Dynamische Speicherverwaltung

Lebensdauer und Sichtbarkeit von dynamischen Variablen

- Lebensdauer:
  Dynamische Variable existieren bis sie explizit gelöscht werden, bzw. bis zum Ende des Programmlaufs.
- Sichtbarkeit:
  Das Konzept Sichtbarkeit greift nicht bei dynamischen Variablen, da sie nicht über Namen, sondern nur über Pointer angesprochen werden.

Dynamische Speicherverwaltung

Bibliotheksfunktionen für dynamische Speicherverwaltung:

```
#include <stdlib.h>

void *malloc(size_t size);

void free(void * ptr);
```

(Der Datentyp `size_t` entspricht im allg. dem Datentyp `unsigend int` oder `unsigend long`)

Speicherverwaltung: malloc()

Die Bibliotheksfunktion **`malloc()`**

Aufruf-Parameter:

- Größe des Speicherbereichs, den die neue Variable benötigt.

Rückgabewert:

- Void-Pointer, der auf die neue, anonyme Variable zeigt.

Speicherverwaltung: malloc()

Die Bibliotheksfunktion `malloc()`

Die Größe des Speicherbereichs, den die neue Variable benötigt, kann mit dem Operator `sizeof` bestimmt werden.

Der zurückgegebene Pointer muss einer Pointer-Variablen zugewiesen werden.

Der Typ des Pointers muss im Allgemeinen nicht explizit konvertiert werden. Manche Compiler verlangen dies aber.

malloc() und free()

```
int main(void){

    int *intPtr = NULL;
    int i;
    int iterations;

    printf("\n Wieviele Zyklen? ");
    scanf(" %d", &iterations);

    for(i = 0; i < iterations; i++){
       intPtr = malloc(sizeof (int));
       *intPtr = i*i;
       printf("\n intPtr zeigt auf: %d", *intPtr);
       free(intPtr);
    }
    return 0;
}
```

Speicherverwaltung: free()

Dynamische Variable, die nicht explizit freigebenen werden, existieren weiter, auch wenn auf sie nicht mehr zugegriffen werden kann (Memory-Leak).

Eine dynamische Variable muss daher freigegeben werden, bevor die letzte Zugriffsmöglichkeit verloren geht. Sie darf nur genau einmal freigegeben werden.

Aufruf z.B.: `free(varPtr);`

Wobei varPtr auf die freizugebende Variable zeigt.

Speicherverwaltung: free()

- Nachdem eine dynamische Variable mit free() freigegeben wurde, sollten (müssen) alle Zeiger, die auf diese Variable zeigen mit einem anderen Wert belegt werden.
- Der Zugriff auf die freigegebene Variable kann zu beliebige Ergebnisse führen. (Wie der Zugriff auf eine nicht initialisierte Variable)

```
free(varPtr);

x = *varPtr; /* Ergebnis ist quasi zufällig */
```

Verkettete Liste/Stack

- Verkettet Liste
  - Menge von dynamischen Variablen linear verkettet
  - Einfügen und Entfernen von Elementen an beliebiger Stelle
- Stack
  - Menge von Variablen
  - Operationen push() und pop()
  - Implementierung z.B. verkettete Liste

Abstrakter Datentyp ADT

- Auf die Werte, die in einem ADT enthalten sind, kann nur über Zugriffsfunktionen zugegriffen werden. Z.B. Einfügen, Löschen

- Die Implementierung ist in einem Teil des Programms gekapselt.
  - Definition der Datenstruktur
  - Implementierung der Zugriffsfunktionen

Stack, nicht abstrakt

- Globale Variable, die auf die Spitze des Stacks zeigt, z.B. `stackTop`
- Globale Datenstruktur mit Verkettungs-Pointer
- Operationen push() und pop()

Nachteil:

- Die Datenstruktur kann an jeder Stelle des Programms geändert also auch gestört werden.

Abstrakter Datentyp Stack

Die interne Struktur des Stacks ist nur innerhalb der Implementierungs-Datei sichtbar.

Im übrigen Programm sind nur Zugriffsfunktionen sichtbar:

- push()
- pop()
- isEmpty()
- . . .

Abstrakter Datentyp Stack

Header-Datei für den Stack:

```
struct order{
    unsigned int orderCode;
    unsigned int price;
};
typedef struct order ORDER;
int isEmpty();
void push(ORDER);
ORDER pop();
```

Abstrakter Datentyp Stack

Vorteil:

- Die Datenstruktur kann nur innerhalb der Implementierungs-Datei geändert werden.
- Speicherverwaltung (`malloc(), free()`) nur innerhalb der Implementierungs-Datei

Ziel:

- Minimierung der Abhängigkeiten zwischen verschiedenen Programmteilen

Dauerhafte Speicherung von Daten

- Variable eines Programms verlieren ihren Wert spätestens, wenn das Programm beendet wird
- Um Daten dauerhaft zu speichern, können sie in Dateien abgelegt werden
- Die Bibliothek `stdio.h` stellt Datei-Funktionen zur Verfügung

Streams

- In C stellen „Streams“ die Verbindung zu einer Datei her (Bibliothek `stdio.h`).
- Ein Programm kann von einem Stream lesen und auf einen Stream schreiben.
- Das „andere Ende“ eines Streams ist i.allg. eine Datei, kann aber auch die Tastatur bzw. der Bildschirm sein (oder ein anderes I/O-Gerät).
- Der Datentyp eines Streams ist: FILE *
  z.B. `FILE *varStream;`

Streams und Dateien

```
int main(void){
   FILE *inFilePtr;
   char line[100];
   char *getResult;

   inFilePtr = fopen("testFile.txt", "rt");
   if(inFilePtr == NULL){
     exit(1);
   }
   while( !feof(inFilePtr) ){
     getResult = fgets(line, 100, inFilePtr);
     if (getResult != NULL){
        printf(" Zeile gelesen: %s ", line);
     }
   }
   fclose(inFilePtr);
   return 0;
}
```

Dateizugriff

- Variable für Stream vereinbaren.
- Datei öffnen und mit einem Stream verknüpfen.
- Von Stream lesen bzw. auf Stream schreiben.
- Datei schließen.

- Nach jedem Aufruf einer Bibliotheksfunktion sollte (muss) geprüft werden, ob der Aufruf erfolgreich war (s. `malloc()`).

Dateizugriff - fopen()

fopen(Dateinamen, Modus)

Der Modus besteht aus ein bis drei Buchstaben.

- Erster Buchstabe (notwendig): 'r', 'w' oder 'a' für read, write bzw. append
- Als zweiter bzw. dritter Buchstabe kann '+' und einer der Buchstaben 'b' oder 't' stehen.

Bsp.: „r+b“, „w“, „a+“, „w+t“

Dabei steht 'b' für binär-Modus und 't' für text-Modus. Das '+' bedeuted, dass die Datei sowohl gelesen als auch beschrieben werden kann.

Dateizugriff – fclose()

```
fclose(FILE * myStream);
```

Der Aufruf fclose(myStream) schließt den Stream. Datenstrukturen im Prozess und im Betriebssystem werden freigegeben.

- von myStream nicht mehr gelesen werden
- auf myStream kann nicht mehr geschrieben werden

Dateizugriff – fopen()

Das Öffnen einer Datei erzeugt *keine* Sperre.

Eine Datei kann von mehreren Prozessen gleichzeitig geöffnet sein.

Wenn mehrere Prozesse auf eine Datei schreiben, müssen sie sich synchronisieren.

Streams: Lese-/Schreib-Funktionen

- fscanf(), fprintf()
  wie scanf() bzw. printf() für beliebige streams
- getc(), putc() bzw. fgetc(), fputc()
  Lesen bzw. schreiben einzelner Bytes
- fgets(), fputs()
  Lesen bzw. schreiben von (Text-)Zeilen
- fread(), fwrite()
  Lesen bzw. schreiben von Datenblöcken

Die genaue Funktionsweise entnehmen Sie bitte den Arbeitsblättern bzw. der Literatur.

Dateizugriff – Positionszeiger

Zu jeder geöffneten Datei gehört ein Positionszeiger. Jede Lese- oder Schreib-Operation findet an der Stelle statt, auf die der Positionszeiger zeigt.

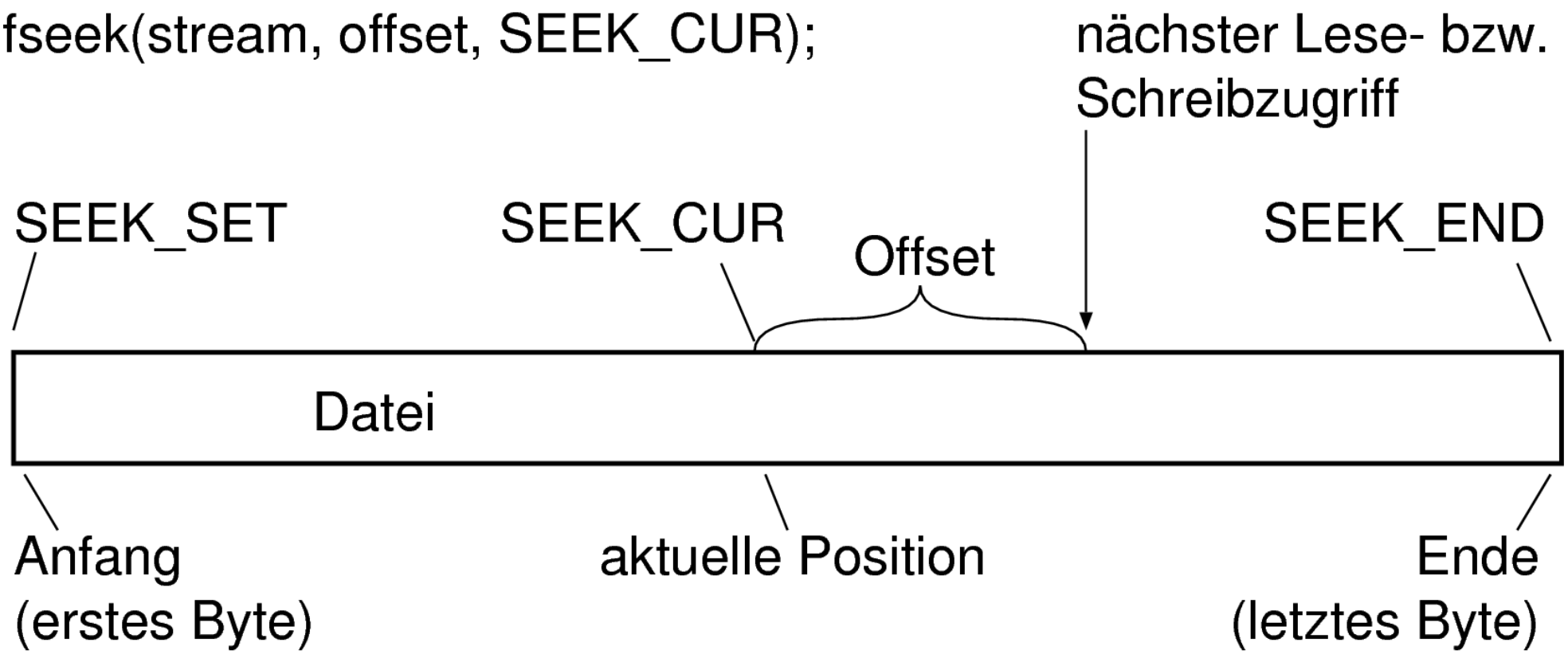
Wert ermitteln: `long ftell(FILE *stream)`

Der Positionszeiger wird durch Funktionsaufrufe verändert:

- Lese- oder Schreib-Operationen
- `fseek(FILE* stream, long offset, int posId)`
  (posId: SEEK_SET, SEEK_CUR oder SEEK_END)
- `rewind(FILE* stream)`

Dateizugriff – Positionszeiger

fseek(stream, offset, SEEK_CUR);



Unterstützung

- Tutoren im Praktikum
- Laborsprechstunde
  Herr Drotleff, T007
  Herr Bernhard, T109
- Zusatzkurs der Fachschaft

Beliebte Missverständnisse

- Null-Byte bei Strings: '\0' oder 0 nicht NULL
- Allgemeine Arrays besitzen i. Allg keine Endemarkierung
- Initialisierung und Vergleich bei Pointer =NULL nicht 0 oder '\0'
- Zugriff auf Komponenten einer Struktur: **`var.comp`** wenn **`var`** kein Pointer ist. **`var->comp`** wenn **`var`** ein Pointer ist. Der Datentyp von **`comp`** spielt keine Rolle. **`comp`** ist der *Name* der Komponente, nicht der *Datentyp*.

Beliebte Missverständnisse

- `File *stream` ≠ Positionszeiger
- Parameterübergabe
  - alle Datentypen außer Arrays: „by value“ (auch Strukturen ggf. mit Arrays)
  - alle Datentypen können „by reference“ übergeben werden (formaler Parameter: Pointer auf . . .)

Fakultät

```
double fakultaet(unsigned short number){
    double result;
    if (number == 0){
       return 1;
    }
    result = number * fakultaet(number – 1);
    return result;
}

int main(void){
  unsigned short num;
  double fak;

  printf("\n Argument fuer Fakultaet eingeben: ");
  scanf("%u", &num);
  fak = fakultaet(num);
  printf("Die Fakultaet von %u ist %e \n", num, fak);
  return 0;
}
```

Rekursive Funktionen

- Eine Funktion, die sich selbst aufruft, wird rekursiv genannt.
- Prinzipiell kann jede rekursive Funktion in eine iterative Funktion umgewandelt werden.
- Manche Algorithmen lassen sich rekursiv eleganter realisieren als iterativ.

Rekursive Funktionen

- Eine rekursive Funktion muss mindestens einen nicht-rekursiven Ausgang besitzen.
- Vor dem rekursiven Aufruf muss geprüft werden, ob das Ende der Rekursion erreicht ist (dann den nicht-rekursiven Ausgang wählen).
- Anwendungen für rekursive Funktionen sind z.B. Sortieren und rekursive Datenstrukturen wie Listen und Bäume.

Enumerations/Aufzählungstypen

```
enum apple {cox, elstar, ontario, golden, klar};
enum pear  {william, helene, most};
typedef enum apple APPLE;

int main(void){
    APPLE myApple = elstar;
    enum pear myPear = william;
    printf("\n\n meine Birne: %d \n", myPear);
    if (myApple == ontario){
       printf("\n meine Apfel ist ein Ontario\n");
    }
    else{
       printf("\n meine Apfel ist ein %d \n", myApple);
    }
    :
```

Enumerations/Aufzählungstypen

Jeder Wert eines Aufzählungstyps entspricht einer ganzen Zahl.

Die Anweisung

```
enum color {red, green, blue};
```

definiert die drei Werte:

`red = 0, green = 1` und `blue = 2.`

Die Werte können gesetzt werden:

```
enum color {red, green = 4, blue};
```

`red = 0, green = 4` und `blue = 5.`

Enumerations/Aufzählungstypen

Die Anweisung

```
enum color {red, green, blue};
```

definiert den Datentyp `enum color` mit drei Werten: `red, green` und `blue.`

Die Anweisung

```
enum color myColor;
```

definiert eine Variable `myColor` vom Typ `enum color.`

Enumerations/Aufzählungstypen

Variable eines Aufzählungstyps können für einen ganzahligen Wert/Ausdruck eingesetzt werden.

```
enum color {red, green, blue};
enum color myColor = green;

float test[3];
test[myColor] = 4.2;

switch(myColor){
  case red: . . .
  case green: . . .
  . . .
}
```

Enumerations/Aufzählungstypen

Variable verschiedener Aufzählungstypen können in einem Ausdruck untereinander und mit Zahlen gemischt werden: schlechter Stil!

```
enum color {red, green, blue};
enum direction {north, west, east = 6, south};

enum color myColor = green;
enum direction myDir = south;

int nonsense = (myDir – myColor) * 3;
if (myColor > myDir){
    . . .
}
```

Enumerations

Sinnvoller Einsatz:

z.B. Darstellung eines Zustands

- Vergleich auf Gleichheit
  - `if (myColor == blue) . . .`
  - `switch (myColor){`
    `case blue : . . .`
- evtl. Indizierung von Arrays:
  - `colorCode = codeTab[blue]`

Der Präprozessor

Übersetzungsschritte eines C-Programms:

(1) Präprozessor (Textverarbeitung):
Kopiert und ersetzt Programmtext

(2) Compiler: Analysiert Programmstruktur und
erzeugt “Objekt-Code“

(3) Linker: Fügt Objekt-Code von
Bibliotheksfunktionen ein und setzt Adressen ein.

Präprozessor-Anweisungn

- Zeilenorientiert:
  Eine Anweisung endet am Ende der Zeile.
  Ausnahme:
  Wenn der letzte Buchstabe der Zeile ein \ ist.
- Erstes (druckbare) Zeichen muss ein # sein
- Beispiele:

```
#include . . .
#define . . .
#ifdef . . .
```

Präprozessor-Anweisungen

```
#include <name.h>
```

Sucht im „include-Pfad“ nach der Datei name.h und ersetzt die include-Anweisung durch den Inhalt der Datei name.h

```
#include “name.h“
```

Sucht im aktuellen Verzeichnis nach der Datei name.h

Schutz gegen mehrfachen Import:

```
#ifndef NAME_H
#define NAME_H
Inhalt der Datei name.h
#endif
```

Defines

Der Präprozessor ersetzt Konstanten im Programm.

```
#define LENGTH 64
```

LENGTH ist eine (Text-)Konstante

Sie wird im Programm ersetzt:

```
int numbers[LENGTH];
          :
for(i = 0; i < LENGTH; i++){ ...
```

Wird zu:

```
int numbers[64];
          :
for(i = 0; i < 64; i++){ ...
```

Defines

Sie sollten keine Zahlen (Magic Numbers) in Ihrem Quellcode verwenden.

Besser:

- Präprozessor-Konstante: `#define SIZE 5`

Noch besser, aber nicht immer möglich:

- Compiler-Konstante: `const int size = 5;`
  (in der Header-Datei als `extern` deklarieren)

Makros

Der Präprozessor ersetzt „Makros“ im Programm.

```
#define PROD(A, B) (A)*(B)
```

PROD ist ein Makro; es wird im Programm ersetzt:

```
int size = 3;
int value = 6;
int result;
result = PROD(size, value);
```

wird zu:

```
result = (size) * (value);
```

Makros

- Makros sind etwas schneller als Funktionsaufrufe
- Makros können zu schwer erkennbaren Fehlern führen
- Makros können meist ersetzt werden durch inline-Funktionen

Empfehlung:

- Keine Makros einsetzten
- inline-Funktionen nur wenn nötig

Präprozessor

Hauptanwendungen:

- Definition von Konstanten
- Mehrere Programmvarianten aus einem Quelltext (statt if-Anweisungen)
  - geringere Größe und höhere Geschwindigkeit des ausführbaren Programms
  - schlechtere Lesbarkeit des Quelltexts